

Patrick Turner

CSC 842 – Tool #3 - Analyze malicious activity on DNS Port 53 traffic

Dr. Welu

July 3, 2026

Sniff53

Sniff53 is a lightweight packet sniffer that parses Domain Name System (DNS) packets on port 53 (UDP) to detect malicious activity. These DNS tunneling attacks can create a communication channel that bypasses many firewalls because they don't inspect traffic that traverses port 53. Corporate sized Next-Generation Firewalls (NGFWs) are now performing deep packet inspection. Attackers can communicate with a Command and Control Server (C2) to send remote execution instructions to compromised computers. If a firewall is not watching what is being sent back and forth on port 53 this can happen very easily. This attack can also be used for data exfiltration, possibly sending small pieces of files or stealing credentials from a network. Data exfiltration prevention hardware and software may miss DNS packets if they are only looking at traditional ways of exfiltrating data. Another attack is used to bypass paid Wi-Fi. Iodine and Your-Freedom are tools that can tunnel normal web traffic through port 53 which is usually open on paid Wi-Fi networks. The initial program was developed using Claude AI. The original interaction and python file have been included in the repository.

The problem:

The problem I am addressing is malicious actors using port 53 UDP to communicate in a non-traditional way through a firewall. The problem is important because an organization that doesn't have a corporate next-generation firewall is subject to attacks and data exfiltration through port 53. This tool can solve this issue by capturing packets traversing the network and identifying issues. This tool fills the gap if a small to medium sized business doesn't have a Next Generation Firewall.

Existing tools:

There are Next Generation Firewalls (NGFWs) that will monitor traffic on port 53 for the indicators listed below. Many of the smaller, less expensive firewalls used in small and medium businesses do not have this capability.

High level architecture of the tool:

The tool is written in python which is readily available on multiple platforms. The tool reads log files as well as pcap files that are generated by Wireshark.

The tool has three basic layers: ingest, features, and detect. It parses a log file or pcap file, applies features to identify issues in the requests/responses, and calculates a score based on these factors. The tool looks for five items that may indicate DNS traffic is being abused. They include:

1. Encoded payloads in subdomains. Tunneling tools like Iodine may encode data in base64 or hexadecimal and may use it in the subdomain portion of the request. This will be flagged under “entropy” and “long_subdomain”.
2. C2 beaconing. C2 servers send out DNS queries communicating with clients. No single query looks suspicious, but a sustained pattern is an alarm. This will be flagged under “host_pattern”.
3. High query rate. Tunneling tools launch DNS queries quickly to get as much throughput as possible. These are flagged under the “query_rate” signal.
4. DGA domain cycling. Malware using domain name generating software cycles through hundreds of domain names that don’t exist. These produce a flood of NXDOMAIN - does not exist. These are flagged under “nxdomain”.
5. Suspicious query types. Normal web browsing is mostly A and AAAA record lookups. TXT records are popular among tunneling tools because they can hold large payloads. Other rare types include NULL, ANY, and NAPTR. These are flagged under “txt_type” and “rare_type”.

Each of the detection areas produces a sub score between 0.0 and 1.0. The closer to 0.0 the less possibility there is a threat. The sub scores are multiplied by a weight that reflects how reliable the signal is:

Signal	Weight
Host pattern	40%
Entropy	25%
Subdomain length	15%
Query rate	15%
NXDOMAIN	10%
Query type	10%
Digit ratio	10%

There are attack techniques that use more than one measurement. For example C2 beaconing is measured by `host_pattern` and `query_rate`. Some signals contribute to multiple techniques, and their weights overlap rather than add up to 100%. Overweighting is a deliberate design choice so that queries with several strong signals hit the HIGH severity more reliably.

How to run the tool:

The tool requires no external dependencies; it runs using the Python standard libraries.

The tool has the following options:

```
Python3 sniff53.py test.pcap
```

```
Python3 sniff53.py testlog.log
```

```
Python3 sniff53.py capture.pcap --min-score 0.5
```

```
Python3 sniff53.py capture.pcap --features
```

```
Python3 sniff53.py capture.pcap -summary
```

The first command runs the sniff53 python script with a pcap capture file and produces groups of triggered DNS queries and a summary.

```
patrickturner@PDTUbuntu:~/Desktop/tool3$ python3 sniff53.py test.pcap

Sniff53
source: test.pcap | threshold: 0.4 | window: 60s

► 10.0.0.99 9 flagged queries, peak 0.738
  reason: entropy, host_pattern, long_subdomain, query_rate, txt_type
  _____
05:09:36 0.603 MEDIUM TXT c2VjcmV0IGRhGEgZXhmaWx0cmF0aW9uIHRlc3Q.tunnel.evil.com
05:09:37 0.661 MEDIUM TXT dGhpcyBpcyBhIGxvbmVmcGcGF5bG9hZCBiZWluZW.tunnel.evil.com
05:09:38 0.661 MEDIUM TXT ZXhmaWx0cmF0aW9uIHRlc3Q.tunnel.evil.com
```

```
05:11:09 0.476 MEDIUM TXT chunk00058.data.exfil-fast.com
05:11:09 0.476 MEDIUM TXT chunk00059.data.exfil-fast.com

--- Summary ---
Queries analysed : 524
Flagged          : 78 (14.9%)

HOST          HITS   PEAK   REASON
10.0.0.99      9     0.74   entropy, host_pattern, long_subdomain, query_rate, txt_type
10.0.0.66      6     0.65   entropy, host_pattern, long_subdomain, query_rate
10.0.0.77     15     0.54   entropy, host_pattern, nxdomain, query_rate
10.0.0.55      3     0.51   digit_ratio, entropy, host_pattern, query_rate
10.0.0.88     45     0.48   entropy, host_pattern, query_rate, txt_type

patrickturner@PDTUbuntu:~/Desktop/tool3$
```

The first few lines give the source file name, the current threshold to report on, and the timing window for the queries. The summary lets you know how man packets were tested and how many were flagged as suspicious. It also lists the hosts that were flagged, number of hits per IP address, peak percentage, and the reason.

The second command is similar to the first one. This command is parsing a log file instead of a pcap file.

```
patrickturner@PDTUbuntu:~/Desktop/tool3$ python3 sniff53.py testlog.txt

Sniff53
source: testlog.txt | threshold: 0.4 | window: 60s

► 10.0.0.99 9 flagged queries, peak 0.661
reason: entropy, host_pattern, long_subdomain, query_rate, txt_type

10:02:01 0.415 LOW TXT c2VjcmV0IGRhdGEgZXhmaWx0cmF0aW9uIHRlc3Q.tunnel.evil.com
10:02:02 0.553 MEDIUM TXT dGhpcyBpcyBhIGxvbmcmcGF5bG9hZCBiZWluZW.tunnel.evil.com
10:02:03 0.553 MEDIUM TXT ZXhmaWx0cmF0ZWQgb3ZlciBkbmMgdHVubVs.tunnel.evil.com
10:02:04 0.661 MEDIUM TXT dGVzdHBheWxvYWQxMjM0NTY3ODkwYWJjZGVm.tunnel.evil.com
10:02:05 0.661 MEDIUM TXT cGF5bG9hZGNodW5rbnVtYmVyc2l4dGVlbG.tunnel.evil.com
10:02:06 0.661 MEDIUM TXT ZmluYWxjaHVua29mZGF0YWJlaW5nc2VudA.tunnel.evil.com
10:02:07 0.661 MEDIUM TXT bW9yZWRhZGFiZWluZ3R1bm5lbGVkb3V0bm93.tunnel.evil.com
10:02:08 0.661 MEDIUM TXT c3RpbGxnb2luZ3dpdGhtb3JlZW5jb2RlZGRhdGE.tunnel.evil.com
10:02:09 0.661 MEDIUM TXT bGFzdGNodW5rb2ZleGZpbHRyYXRlZGRhdGE.tunnel.evil.com
```

The third command uses a min-score switch to change what the tool triggers on and makes it more sensitive.

```
patrickturner@PDTUbuntu:~/Desktop/tool3$ python3 sniff53.py test.pcap --min-score 0.2
```

Sniff53
source: test.pcap | threshold: 0.2 | window: 60s

► 10.0.0.99 10 flagged queries, peak 0.738
reason: entropy, host_pattern, long_subdomain, query_rate, txt_type

05:09:35	0.365	LOW	TXT	aGVsbG8gd29ybGQgdGhpcyBpcyBhIHRlc3Q.tunnel.evil.com
05:09:36	0.603	MEDIUM	TXT	c2VjcmV0IGRhdGEgZXhmaWx0cmF0aW9uIHRlc3Q.tunnel.evil.com
05:09:37	0.661	MEDIUM	TXT	dGhpcyBpcyBhIGxvbmcmcGF5bG9hZCBiZWluZW.tunnel.evil.com

The fourth command uses -- feature which adds what triggered each DNS query.

```
patrickturner@PDTUbuntu:~/Desktop/tool3$ python3 sniff53.py test.pcap --feature
```

Sniff53
source: test.pcap | threshold: 0.4 | window: 60s

► 10.0.0.99 9 flagged queries, peak 0.738
reason: entropy, host_pattern, long_subdomain, query_rate, txt_type

05:09:36	0.603	MEDIUM	TXT	c2VjcmV0IGRhdGEgZXhmaWx0cmF0aW9uIHRlc3Q.tunnel.evil.com	entropy=4.50 sub_len=46 qrate=0.05/s nxdom=0.00 TXT=yes
05:09:37	0.661	MEDIUM	TXT	dGhpcyBpcyBhIGxvbmcmcGF5bG9hZCBiZWluZW.tunnel.evil.com	entropy=4.20 sub_len=45 qrate=0.08/s nxdom=0.00 TXT=yes
05:09:38	0.661	MEDIUM	TXT	ZXhmaWx0cmF0ZWQgb3ZlciBkbnMgdHVubmVs.tunnel.evil.com	entropy=4.41 sub_len=43 qrate=0.12/s nxdom=0.00 TXT=yes

Finally, the last command uses the -- summary feature which shows just the summary.

```
patrickturner@PDTUbuntu:~/Desktop/tool3$ python3 sniff53.py test.pcap --summary
```

— Summary —

Queries analysed : 524
Flagged : 78 (14.9%)

HOST	HITS	PEAK	REASON
10.0.0.99	9	0.74	entropy, host_pattern, long_subdomain, query_rate, txt_type
10.0.0.66	6	0.65	entropy, host_pattern, long_subdomain, query_rate
10.0.0.77	15	0.54	entropy, host_pattern, nxdomain, query_rate
10.0.0.55	3	0.51	digit_ratio, entropy, host_pattern, query_rate
10.0.0.88	45	0.48	entropy, host_pattern, query_rate, txt_type

```
patrickturner@PDTUbuntu:~/Desktop/tool3$
```

Both the pcap file and the log file are included in the repository.

Tool testing:

The tool has been tested with an AI generated pcap and log file. Both are included in the repository. I tried using Wireshark on a corporate network, but it is rare that you get any of these hits in a reasonable amount of time.

Results:

The results were as expected. I found it difficult to get a set of data from a live capture so I have included the test files.

Known issues

The current state of the tool runs on captured data, either from a pcap file or log file. It currently does not watch live traffic traversing the network. That option will probably be added as a revision at a later date.